DAYBREAK — Hackathon Build Plan

Status: proposed implementation plan, not a claim of completed work.
Working name: Daybreak. Brand/name availability has not been checked.
Goal: a finite, beautifully edited daily newspaper that understands the reader, teaches something memorable, and improves with use.

1. Product thesis

Your morning, edited for you. Deliver one coherent daily edition rather than an infinite recommendation feed. Personalize story selection, ordering, depth, and section emphasis while preserving source attribution and an immutable published edition.

North-star demo: two readers receive visibly different papers from the same real source pool; feedback changes a subsequent edition; the downloaded PDF contains the same editorial content as its web edition.

Planning assumptions: hackathon-sized team and limited time; exact deadline, budget, provider keys, team size, and deployment access are unknown. Sequence below is mandatory; time estimates are illustrative, not promises.

2. Requirement coverage and proof

Brief requirement	Planned implementation	Acceptance evidence
Onboarding captures interests with good UX	Short guided flow: free-text interests, editable topic chips, goals, region, time budget and timezone; preview and defaults	A new reader reaches a personalized edition without completing a long questionnaire
Gather relevant articles across the web	Server-side ingestion from multiple permitted feeds/APIs, with timestamps, attribution, canonical URLs and deduplication	Edition contains real stories from at least three working sources when available; insufficient coverage is disclosed
Fresh daily newspaper PDF	Scheduled daily edition builder; stored PDF; deliberate newspaper layout	Two dated editions; functioning scheduled job; readable multi-page PDF
Personalize content and presentation	Topic/entity weights, region, reading depth, reading budget and section ordering	Two profiles generate meaningfully different selections and layouts
Complete webpage reading experience	Inline editorial summaries and explainers, sections, source links, saved/read states and completion	Reader can understand the edition without opening every publisher page
Click original / skip unwanted stories	Per-story original link, skip, more/less like this and reason controls	All links resolve to stored source URLs; feedback persists
Same edition on web and PDF	One persisted edition snapshot drives two renderers	Same story IDs, order, copy, sources and edition version in both
Access previous papers	Edition archive with date cards and historical web/PDF views	Yesterday's paper remains unchanged after preferences change

Brief requirement	Planned implementation	Acceptance evidence
Improve with use	Explicit feedback updates an inspectable interest profile; passive signals carry less weight	Feedback creates a profile change and affects the next edition
Room to explore	Context Lens, Since You Last Read, Recall Three	At least one polished, grounded differentiator works end-to-end
Repository at hackathon start	Participant creates own repo immediately; small genuine commits throughout	Real commit history; public repository at submission
Submission	Public repo, Loom <= 5 minutes, architecture diagram	Checklist verified before submission

The brief does not prescribe a framework, story count, team size, or scoring rubric. These are our choices, not organizer requirements.

3. Scope and priority

P0 — must ship

- Authentication or a clearly isolated persistent reader identity.
- Short onboarding with editable extracted preferences.
- Real multi-source ingestion and source attribution.
- Deduplicated, ranked daily edition with useful inline summaries.
- Web reader, original links, skip, save, more/less feedback.
- Print-designed downloadable PDF, not a raw dashboard screenshot.
- Daily scheduling, persisted editions, archive and retry-safe generation.
- Visible feedback-driven profile updates affecting future editions.
- Responsive UI, accessibility basics, loading/error/empty states.
- README, architecture diagram, reproducible setup and submission assets.

P1 — differentiated product

1. Context Lens: 'Why this matters to you' grounded in chosen goals and source evidence. Never fabricate personal facts or causal claims.
2. Since You Last Read: compare today's supported facts with stories in the reader's last relevant edition. Say 'new since your last edition', not 'the only things that changed'.
3. Recall Three: a finish card with three takeaways and optional short questions. Answers link back to edition story IDs. PDF includes takeaways, not interactive controls.
4. Transparent editor: explain recommendation reasons and let readers edit the interest profile.

Ship Context Lens first; add the others only once P0 is stable.

P2 — stretch only

- Ask Your Paper: retrieval restricted to the edition's permitted source material; citations and explicit insufficient-evidence responses.
- Deliberate discovery slot, visibly marked, with user control.
- Optional audio briefing if generation and playback are already reliable.
- Shareable excerpt with permission-aware access; never expose a private edition by default.

Do not build social feeds, autonomous browsing agents, custom ML training, mobile apps, publisher paywall bypasses, or multiple competing backends.

4. Reader experience

Onboarding: target roughly 60–90 seconds; validate in testing

1. 'What are you curious about?' Natural-language input plus suggested topic chips.
2. AI proposes topics, entities and goals; reader confirms/edits. Keep a deterministic chips-only fallback.
3. Pick reading time: 5, 10 or 15 minutes; region and timezone with editable defaults. Optional expertise/depth and source exclusions.
4. Show an honest preview of section emphasis. Generate the first edition and display progress states tied to actual work.

Do not infer political affiliation, health status, or other sensitive attributes from reading behavior. Do not demand profession, address, or lengthy personal data.

Daily reader

- Masthead, date in reader timezone, edition number, reading-time estimate and PDF action.
- Lead story; concise personal relevance line; supporting coverage and visible sources.
- Topic sections, short briefs, one optional discovery slot, and end-of-edition takeaways.
- Expandable background or glossary without replacing the saved edition text.
- Calm controls: original article, save, more, less, skip. A plain skip is not automatically dislike.
- Archive and an editable 'My editor' profile.
- Finite completion state: 'You are caught up for this edition.'

Screens/routes: landing, onboarding, today's edition, archive, historical edition, preferences; authenticated print route; optional edition-scoped chat panel.

5. Visual direction

Editorial rather than dashboard-like. Warm paper (#F5F1E8), near-black ink (#191919), one rust accent (#A84432), restrained rules and generous spacing. Proposed values; contrast must be checked.

Use a locally bundled or properly licensed display serif for headlines and a readable sans-serif for controls. Limit font weights and avoid remote font dependencies during PDF generation.

Desktop: lead story hierarchy, 2–3-column editorial grid, restrained metadata and source marks. Mobile: single column, comfortable type and tap targets; no horizontal newspaper columns. Avoid tiny type, excessive cards, shiny gradients and animation that blocks reading.

Support keyboard navigation, focus states, semantic headings, alt text where needed, reduced motion and non-color-only feedback.

PDF design

- A4 portrait by default; roughly 3–5 pages for a medium edition, depending on content.
- Designed masthead, running date, folio/page number, section labels and source references.
- Page 1: lead + important briefs; interior pages: personalized sections; ending: takeaways and sources.
- Clickable original-source URLs; attribution with publication dates.
- Print-specific layout with deliberate story blocks and page breaks; prevent clipped text, orphaned headings and blank accidental pages.
- Size copy to the selected reading budget, but never cut sentences to fit a page.
- Wait for fonts and required assets before PDF capture. Fall back to text-only if an image fails.
- Same editorial content and ordering as web; responsive geometry and web-only controls may differ.
- Add a concise AI/editorial disclosure and edition timestamp.

6. Recommended implementation

Use one TypeScript application plus one small worker, not a microservice fleet.

- Next.js + TypeScript: application UI, server routes and authenticated print template.
- Tailwind CSS and a small accessible component set: controls; bespoke editorial components for the newspaper.
- Supabase: Postgres, Auth and private storage. Enable row-level security for user-owned data. pgvector is optional, not a prerequisite.
- One LLM provider behind an adapter: structured preference extraction, grounded summaries, contextual explainers, optional embeddings and edition Q&A. Choose the actual model after checking available keys, latency and budget.

- RSS and/or licensed APIs: a curated list of working cross-topic sources, not fragile arbitrary-site scraping.
- Node worker with Playwright/Chromium: print HTML to PDF. Deploy where the browser binary, memory and runtime are known to work; do not assume default serverless compatibility.
- Scheduler: daily trigger to ingestion/edition jobs. For the demo, support one explicitly configured morning timezone first; expand to per-reader schedules if feasible.

Technical verification: Playwright page.pdf uses print CSS and supports preferCSSPageSize. Supabase supports row-level security on Postgres and pgvector. Vercel cron uses UTC, so convert the configured local delivery time or implement due-reader checks if using it.

7. Architecture

```
flowchart TD
  U[Reader] --> UI[Next.js onboarding / reader / archive]
  UI --> API[Authenticated server API]
  API --> DB[(Postgres: readers, preferences, articles, editions, events)]
  S[Scheduler or authorized demo trigger] --> J[Job runner]
  F[Permitted RSS feeds / news APIs] --> I[Ingest and normalize]
  J --> I
  I --> D[Canonical URL dedup + story clustering]
  D --> DB
  DB --> R[Filter + rank + diversity + reading budget]
  R --> L[Grounded LLM editorial generation]
  L --> V[Schema / evidence / source validation]
  V --> E[Immutable edition snapshot]
  E --> DB
  E --> W[Web renderer]
  E --> P[Authenticated print HTML]
  P --> B[Playwright PDF worker]
  B --> O[Private PDF storage]
  UI --> O
  UI --> T[Feedback events]
  T --> Q[Preference updater]
  Q --> DB
```

Implement authorization checks for worker access and signed PDF downloads. Avoid publishing private print URLs or passing long-lived tokens in URLs.

8. Data model

- profiles: user_id, timezone, region, delivery_time, reading_minutes, depth, onboarding_complete.
- preferences: user_id, topic/entity, explicit_weight, learned_weight, origin, updated_at.
- sources: id, name, feed_url, permitted_use_notes, status, last_success_at.
- articles: id, canonical_url, source_id, title, published_at, fetched_at, permitted_excerpt/body, content_hash, topics, optional_embedding, cluster_id, extraction_status.

- editions: id, user_id, local_date, version, state, content_json, profile_snapshot, source_cutoff, generated_at, pdf_status, pdf_storage_key.
- edition_stories: edition_id, article_ids, position, section, relevance_reasons; can be derived from snapshot if avoiding duplicate storage.
- feedback_events: id, user_id, edition_id, story_id, type, optional_reason, timestamp.
- jobs: id, kind, idempotency_key, status, attempts, last_error, timestamps.

Keep shared article metadata separate from private reader data. Limit stored personal information and provide profile reset. Store the published text itself, not just pointers to mutable live summaries.

Unique daily-generation key: user + local_date + version. Repeated scheduler delivery must not generate duplicate public editions. Feedback modifies future editions, not the current immutable snapshot. Explicit regeneration creates a new version.

9. Content pipeline

1. Fetch source updates with timeouts, limits and source health tracking.
2. Normalize URLs and dates; reject empty, malformed or future-dated records.
3. Canonical URL/content-hash dedup; light title/entity clustering to avoid repeated coverage.
4. Use permitted article text when available. If only a feed snippet is accessible, produce a shorter clearly limited brief, not an invented full-article summary.
5. Filter by freshness, language, exclusions and extraction quality.
6. Rank against the confirmed reader profile, with source/topic diversity and a reading-time budget.
7. Generate structured summaries with source IDs and evidence references. Treat source text as untrusted data, never instructions.
8. Validate structure, referenced source IDs, attribution and unsupported names/numbers; these checks reduce errors but do not guarantee factuality.
9. Persist an edition snapshot, render web and PDF from it, and expose generation status.
10. If a model/source fails, use source-attributed available excerpts and disclose limitations. If no fresh valid stories exist, say so rather than invent news or relabel old news as fresh.

Avoid a giant agent loop. Batch article work, cache shared article summaries, and personalize selection/relevance lines rather than re-summarizing every source for every reader.

10. Personalization and learning

Initial ranking heuristic, to tune rather than market as scientifically validated:

score = 0.45 * interest_match + 0.20 * freshness + 0.15 * goal_match + 0.10 * region_match + 0.10 * source_preference - repetition_penalty

Normalize features before combining them. If a feature is unavailable, renormalize available weights rather than assigning arbitrary certainty. Source preference is not a claim of objective

source reliability.

After ranking, apply source/topic caps and optionally reserve about 10% of the reading budget for opt-in discovery. Story-count targets are flexible: roughly 6–10 for a medium edition, subject to length and real coverage.

Signal interpretation:

- Explicit more/less and editable interests: strongest.
- Save: useful positive signal, not proof of endorsement.
- Meaningful dwell/expansion: weak positive signal only while the tab is active.
- Scroll alone: very weak, never treated as agreement.
- Skip: ambiguous; optionally ask 'already know this', 'not interested', or 'not now'. Only topic dislike reduces interest.
- Repeated explicit feedback adjusts learned weights within limits; explicit preferences remain editable and dominant.

Deduplicate events, cap repeated same-story influence, decay older learned signals and support reset. Explain changes in plain language. Demonstrate learning with an honestly labeled subsequent-edition preview; do not pretend a day elapsed.

11. AI contracts

Preference extraction output: topics[], entities[], goals[], suggested_depth, clarification_needed. Reader approves edits.

Article editorial output: headline, summary, key_facts[{text, source_ids, evidence_reference}], background, source_ids, limitations[]. Model cannot invent a source ID.

Edition story output: stable story_id, section, order, article_ids, summary, why_for_you, key_takeaway, source_links.

For Context Lens, distinguish source-backed facts from a modest explanation of relevance to the user's stated goals. Avoid investment, health, or other consequential personalized advice.

For Ask Your Paper, answers must reference only edition materials. A request requiring outside facts receives an explicit limitation. No access to other users' profiles or editions.

12. Build sequence and checkpoints

Phase 0 — repository and foundations (~5% of effort)

Participant creates repo before coding. Commit starter, this plan, .env.example, lint/build setup and minimal architecture. Check available keys and hosting early. Never commit secrets.

Phase 1 — vertical slice (~20%)

One real source -> normalized article -> stored edition -> web reading -> one correctly rendered PDF. Prove deployment and Chromium now, not at the end.
Exit: deployed authenticated reader can download a PDF of its saved edition.

Phase 2 — all core requirements (~35%)

Add remaining sources, onboarding, profile-based ranking, multi-story layout, feedback, subsequent generation, archive and real scheduling.
Exit: every P0 requirement has a working demo path. No extra AI features before this gate.

Phase 3 — design and differentiation (~20%)

Refine typography, mobile behavior, PDF pagination, progress/error states and Context Lens. Add Since You Last Read and Recall Three only if stable.
Exit: two personas visibly differ; no clipping in PDF; feedback loop works.

Phase 4 — reliability and submission (~20%)

End-to-end tests, privacy checks, fresh data, demo rehearsal, README, diagram export, <=5-minute Loom, public repository and submission verification.
Freeze features before recording. If schedule slips, cut P2, then optional P1; never cut PDF/archive/learning/scheduling.

Parallel ownership, if team size allows

- Product/frontend: onboarding, reader, archive, preferences, responsive styling.
- Content/backend: ingestion, database, ranking, feedback, scheduling.
- Editorial/AI: structured outputs, grounding, Context Lens, recall and fixtures.
- PDF/QA/release: print renderer, deployment, tests, docs and demo recording.

For 1–2 people, merge roles and ship only one P1 feature.

13. Acceptance checklist

- [ ] Fresh signup completes onboarding; extracted topics are editable.
- [ ] Two sharply different test personas receive different ranked editions.
- [ ] Edition includes attributed real multi-source content and working original links.
- [ ] Inline reading is useful without clicking through every story.
- [ ] Skip/save/more/less persist and do not corrupt the edition snapshot.
- [ ] Preference changes demonstrably affect a future edition.
- [ ] Scheduled fresh generation works; repeated job is idempotent.
- [ ] Timezone boundary does not create duplicate or wrongly dated editions.
- [ ] Historical editions and historical PDFs remain accessible and stable.

- [ ] Web and PDF share story IDs, order, editorial copy, sources and version.
- [ ] PDF has no overflow, clipped text, blank accidental pages or broken font glyphs.
- [ ] Long titles, missing images and unusually long summaries render acceptably.
- [ ] Reader A cannot fetch Reader B's edition, events, print view or PDF.
- [ ] Provider/source outage produces an honest retry/fallback state.
- [ ] Empty results do not become fabricated stories.
- [ ] Mobile, keyboard, focus and reduced-motion checks pass.
- [ ] Build and tests pass; no secrets are in repo or browser responses.
- [ ] README, public repo, diagram and <=5-minute Loom are ready.

Suggested automated tests: ranking unit tests; feedback-weight tests; source/schema validation; generation idempotency; edition content-parity checks; authorization integration tests; browser E2E for onboarding -> edition -> feedback -> next edition -> archive -> PDF.

14. Five-minute demo storyboard

0:00–0:25: problem and product promise.
0:25–1:05: quick onboarding with a specific reading goal.
1:05–2:00: personalized front page, source links and Context Lens.
2:00–2:45: download/open actual PDF, showing editorial layout and content parity.
2:45–3:35: feedback, inspect updated interests, generate a clearly labeled next-edition preview.
3:35–4:05: archive and optional Recall Three or Since You Last Read.
4:05–4:40: architecture, actual scheduler/job evidence and reliability safeguards.
4:40–4:55: concise close and working product view.

Use real date labels. Historical demo fixtures must be labeled fixtures; never imply seeded data or manual generation proves scheduled production history.

15. Risks and fallback policy

- Browser deployment failure: prove PDF generation early; use a known-compatible Node worker if serverless packaging fails.
- Thin/paywalled source content: use permitted snippets, shorten output and expose limits; never bypass access controls.
- LLM failure: schema validation, bounded retries, grounded excerpt fallback, visible limits.
- Slow generation: ingest once, cache article work, queue jobs, show truthful progress and retry.
- Excess scope: protect complete core journey; remove audio/chat before reducing core quality.
- Filter bubble: editable profile, source diversity, optional discovery; do not optimize solely for clicks.
- False learning demo: persist feedback and show actual deterministic ranking changes.

- IP/privacy: attribution is not a substitute for permission. Check source terms, minimize retained full text, never reproduce restricted articles without authorization.

16. Immediate next actions

1. Participant creates repository at the hackathon start and commits the starter/plan.
2. Confirm remaining hours, team ownership, existing stack, provider keys and hosting budget.
3. Lock the single-stack architecture and one P1 differentiator.
4. Prove real article -> saved edition -> web -> PDF on the target deployment.
5. Complete requirement coverage before expanding scope.

Source basis

- Organizer brief: NaSCon-26-Problem-Statement.pdf, supplied by the participant, pages 1–2.
- Playwright PDF API: https://playwright.dev/docs/api/class-page
- Supabase RLS / vector permissions: https://supabase.com/docs/guides/ai/rag-with-permissions
- Vercel cron documentation: https://vercel.com/docs/cron-jobs

Technical docs support the named platform capabilities; product scope, design, scoring weights and schedule allocation are proposed decisions, not externally established facts. Recheck deployment limits and source permissions during implementation.